Introduzione

Base di Dati (BD) o database: insieme strutturato e organizzato di dati omogenei utilizzati per il supporto allo svolgimento di attività (ente, azienda, ufficio, persona).

Database Management System (DBMS): sistema software progettato per consentire la creazione, manipolazione e interrogazione di uno o più database in modo corretto ed efficiente.

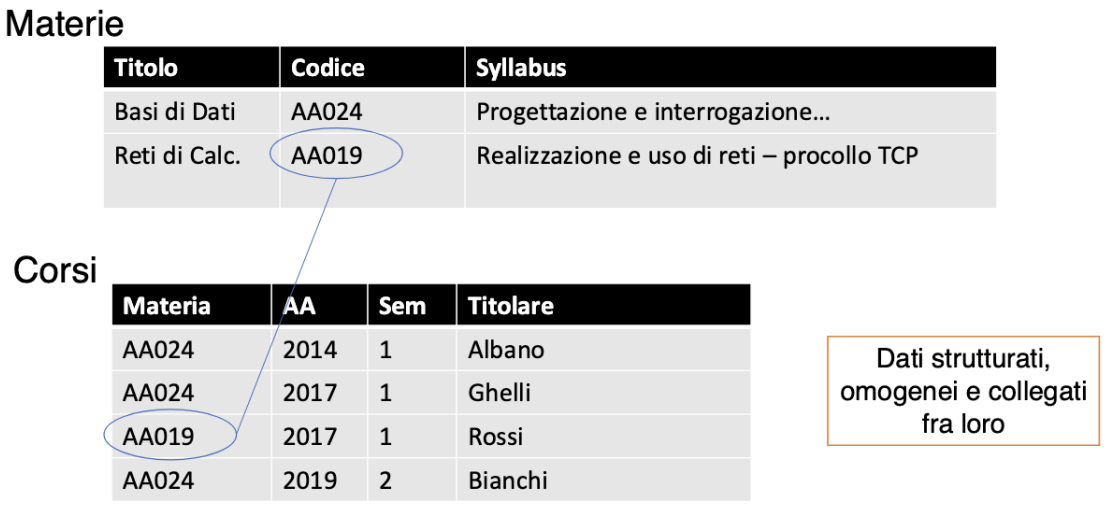


Figure coinvolte:

Committente

- Dirigente
- Operatore

Fornitore

- Direttore del progetto
- Analista

- Progettista di BD
- Programmatore di applicazioni che usano BD

Manutenzione e messa a punto della BD

- Gestione del DBMS
- Amministratore del DBMS

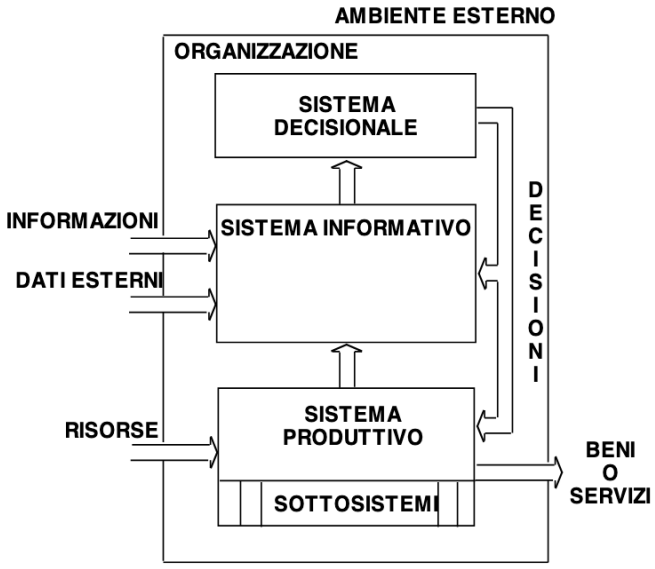
Le basi di dati hanno quindi un ruolo nel sistema informativo e nell'organizzazione di un'azienda.

> 💡 Un sistema informativo di un'organizzazione è una combinazione di risorse, umane e materiali, e di procedure organizzate per:
> la raccolta, l'archiviazione, l'elaborazione e lo scambio delle informazioni necessarie alle attività operative o di programmazione e controllo.

Esempi di sistemi informativi:

- Università:
  - Gestione studenti, corsi e docenti
  - Registrazione esami e voti
  - Prenotazione aule e laboratori
- Ospedale:
  - Gestione pazienti e cartelle cliniche
  - Prenotazione visite ed esami

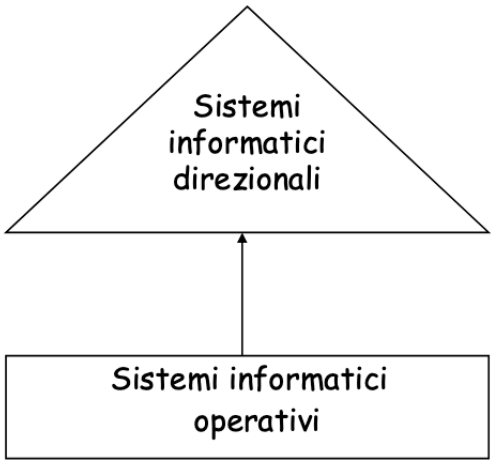
    - Gestione personale medico e turnazioni
- Banca:
    - Gestione conti correnti e operazioni
    - Concessione prestiti e mutui
    - Analisi del rischio clienti



In tutti questi esempi, il sistema informativo include sia componenti informatizzate (database, software) che componenti non informatizzate (persone, procedure operative, regolamenti).

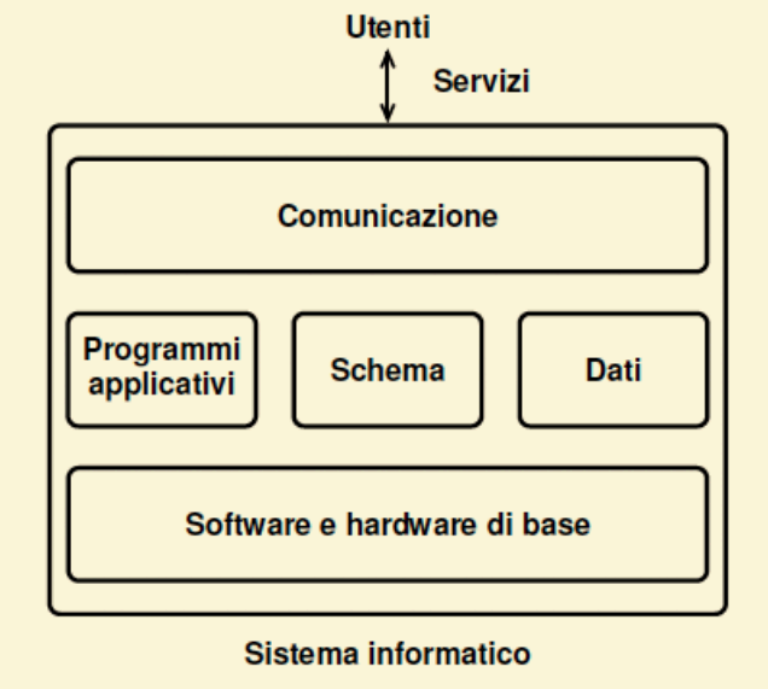
Sistemi informatici vs Sistemi

- **Sistema informativo automatizzato:** è la parte del sistema informativo che non si limita a usare carta, penna e cervelli umani, ma si appoggia a un **sistema informatico** per raccogliere, elaborare, archiviare e scambiare informazioni.
- **Sistema informatico:** è l'insieme delle tecnologie informatiche e di comunicazione
  (ICT: *Information and Communication Technology*) usate per supportare le attività di un'organizzazione. In pratica include hardware, software, reti e persone che li gestiscono.



In modo gerarchico:

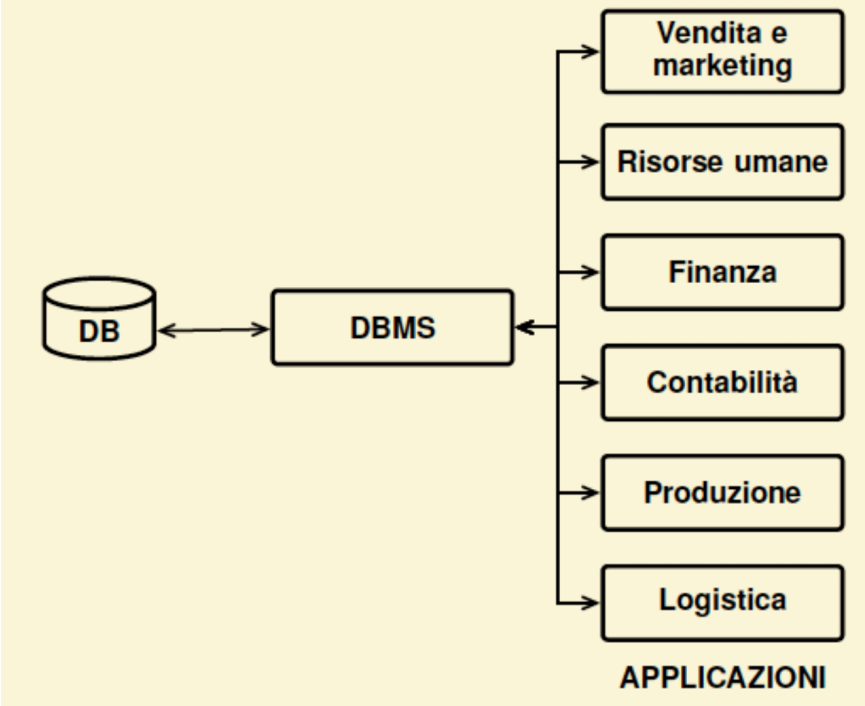
1. ***Sistema informativo*** = tutto ciò che gestisce informazioni in un'organizzazione
2. ***Sistema informativo automatizzato*** = la parte informatizzata del sistema informativo.
3. **Sistema informatico** = la "cassetta degli attrezzi tecnologici" che rende possibile l'automatizzazione.



Alcune sigle:

- **Data processing (*DP*)**
- **Electronic Data Processing (*EDP*)**
- **Transaction Processing Systems (*TPS*)**

Le caratteristiche delle BD sono garantite da un ***DBMS***, che ha il controllo dei dati e li rende accessibili agli utenti autorizzati.



I dati sono organizzati in Data Warehouse (DW) e gestiti da un opportuno sistema.

Le applicazioni, dette di Business intelligence, sono strumenti di supporto ai processi di controllo delle prestazioni aziendali e di decisione manageriale.

DW viene aggiornato periodicamente con nuovi dati storici, mentre un BD operazionale vien aggiornato tempestivamente.

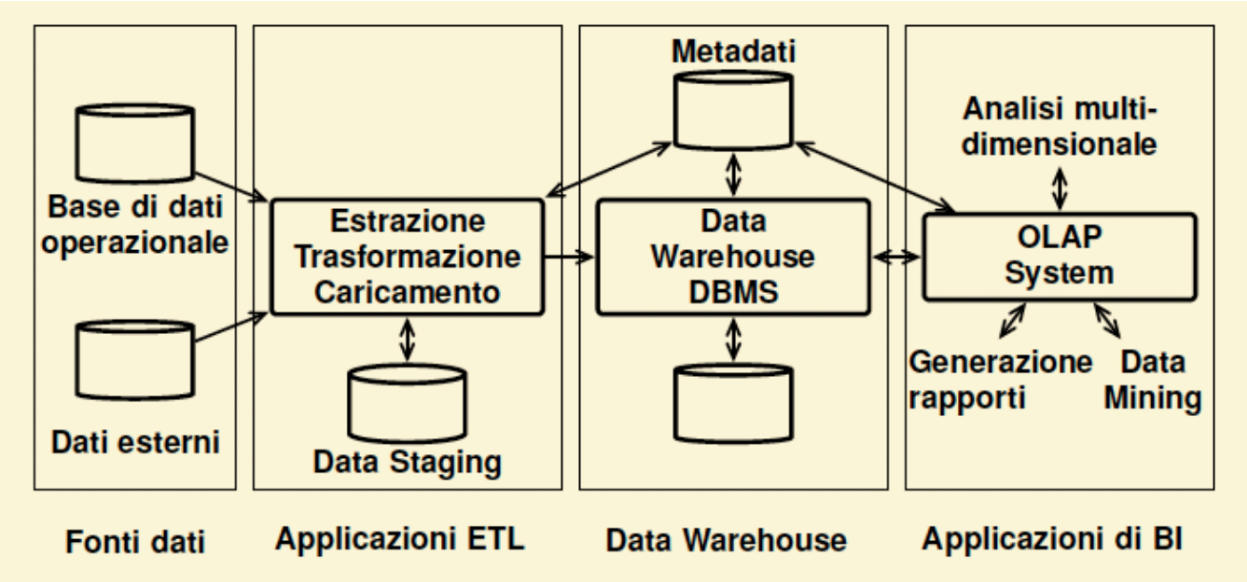
Terminologia:

- Management Information Systems (***MIS***)
- Decision Support Systems (***DSS***)
- Executive Information System (***EIS***)

Uso principale dei DW: OLAP (On-Line Analytical Processing)

- Analisi dei dati di supporto alle decisioni

- Operazioni complesse e non-ripetitive
- Ogni operazione può coinvolgere molti dati
- Dati aggregati, storici, anche non attualissimi



Differenze tra OLTP e OLAP

	OLTP	OLAP
Scopi	Supporto operatività	Supporto decisioni
Utenti	Molti, esecutivi	Pochi, dirigenti e analisti
Dati	Analitici, relazionali	Sintetici, multidimensionali
Usi	Noti a priori	Poco prevedibili
Quantità di dati per attività	Bassa (decine)	Alta (milioni)
Orientamento	Applicazione	Soggetto
Aggiornamenti	Frequenti	Rari
Visione dei dati	Corrente	Storica
Ottimizzati per	Transazioni	Analisi dei dati

Big Data

- Non è solo “tanti dati”, ma dati che crescono oltre i limiti dei sistemi classici (DB relazionali e Data Warehouse).
- Si descrivono con le **3 V**:
  - **Volume** → quantità enorme di dati (terabyte, petabyte e oltre).
  - **Varietà** → dati strutturati (tabelle), semi-strutturati (JSON, XML) e non strutturati (immagini, video, log).
  - **Velocità** → flussi continui di dati che arrivano in tempo reale (sensori IoT, social media, log di sistemi).
- Tecnologie tipiche:
  - **NoSQL** (MongoDB, Cassandra, Redis...) per gestire dati non tabellari.
  - **Data Mining & Machine Learning** per estrarre conoscenza utile.
  - **Data Lake** come “deposito grezzo” dei dati, da cui attingere per successive elaborazioni.

Data Mining

- È una **fase del processo di Knowledge Discovery in Databases (KDD)**, cioè della scoperta di conoscenza dai dati.
- Serve a trovare schemi, relazioni o modelli nascosti che non sono evidenti a prima vista.
- Un **modello** qui è una rappresentazione di pattern significativi (per esempio: clienti che acquistano insieme prodotti A e B hanno alta probabilità di comprare anche C).
- È **interattivo e iterativo**: non basta una sola "corsa" sugli algoritmi, bisogna affinarli, interpretare i risultati, ripetere.

Un esempio concreto:

- **Big Data** → Amazon raccoglie miliardi di click, ricerche, recensioni e acquisti.
- **Data Mining** → su quei dati applica algoritmi di clustering e associazione per suggerire: "Chi ha comprato questo ha comprato anche...".

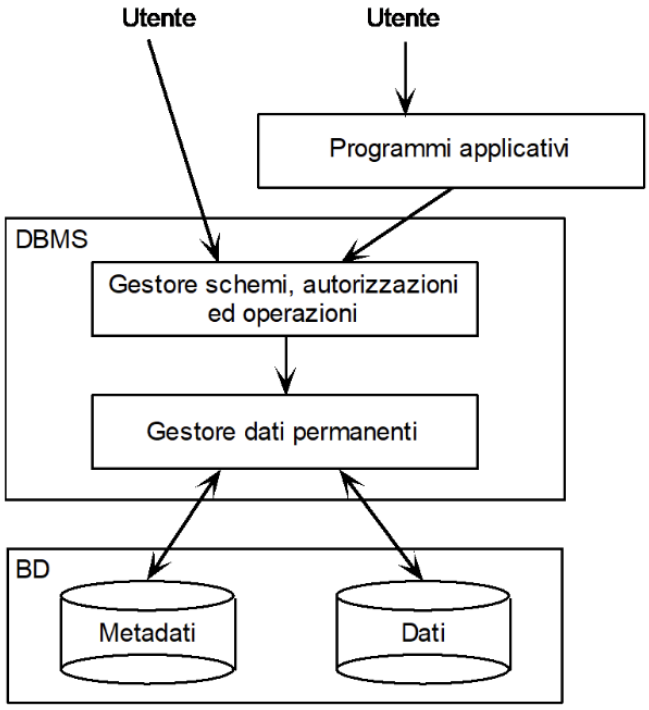
Caratteristiche dei dati in un database

- **Strutturati e relazionati:** i dati non sono mucchi casuali, ma organizzati in insiemi (tabelle, record, ecc.) con relazioni tra loro, definite nello *schema* tramite un modello dei dati (relazionale, gerarchico, a oggetti...).
- **Numerosi:** sono molti, spesso più dei metadati che li descrivono, e non possono stare solo in RAM: serve memoria permanente (dischi, SSD).
- **Persistenti:** rimangono finché non vengono eliminati in modo esplicito. Non spariscono quando si spegne il computer.
- **Accessibili tramite transazioni:** cioè unità di lavoro atomiche: o si completano tutte, o non se ne applica nessuna parte. Questo garantisce consistenza.
- **Protetti:** da accessi non autorizzati e da corruzione accidentale (errori hardware, crash software).

- **Condivisibili:** più utenti possono accedere agli stessi dati contemporaneamente senza distruggerne l'integrità.

Sistemi per Basi di Dati (DBMS)

- Un **DBMS (DataBase Management System)** è il "direttore d'orchestra" che gestisce dati, utenti e applicazioni.
- Può essere **centralizzato** (un server unico) o **distribuito** (più nodi connessi).
- Offre linguaggi e strumenti per:
  - **Definire lo schema** (struttura logica della base di dati).
  - **Scegliere strutture di memorizzazione** (indici, file, B-tree, hash...).
  - **Memorizzare, interrogare, aggiornare i dati** con linguaggi di interrogazione (tipicamente SQL).
  - **Interfacciarsi con programmi** applicativi che usano i dati.



Esempio Sessione DBMS Relazionale

1. Creazione della base di dati

```
create database EsempioEsami;
```

2. Definizione dello schema

```
create table Esami (
    Materia char(5),
    Candidato char(8),
    Voto int,
    Lode char(1),
    Data char(6)
);
```

Qui definisci la tabella **Esami**, specificando gli **attributi** (colonne) e i relativi **tipi di dato**.

- `Materia` è un codice testuale breve (5 caratteri).
- `Candidato` è la matricola dello studente (8 caratteri).
- `Voto` è un intero.
- `Lode` è un campo di 1 carattere (es. “S” per sì, “N” per no).
- `Data` è in formato `char(6)` (probabilmente `GGMMAA` → 07/09/00).

3. Inserimento dei dati

```
insert into Esami
values ('BDSI1','080709',30,'S','070900');
```

Inserisce un record:

- Materia: BDSI1
- Candidato: 080709
- Voto: 30

- Lode: S
- Data: 07/09/00

4. **Interrogazione**

```
select Candidato
from Esami
where Materia = 'BDSI1' and Voto = 30;
```

risultato:

```
Candidato
---------
080709
```

Funzionalità dei DBMS

1. Linguaggi per la definizione della base di dati

Servono a **definire lo schema** della base di dati: quali tabelle esistono, che attributi hanno, quali sono le chiavi, i vincoli di integrità, ecc.

- Esempio: **DDL (Data Definition Language)** in SQL (`CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`).

2. Linguaggi per l'uso dei dati

Sono quelli che permettono di **inserire, aggiornare, cancellare e interrogare i dati**.

- Esempio: **DML (Data Manipulation Language)** (`INSERT`, `UPDATE`, `DELETE`) e **query language** (`SELECT`).

3. Meccanismi per il controllo dei dati

Il DBMS garantisce che i dati siano **consistenti, protetti e affidabili**. Qui rientrano:

- Le **transazioni** e le proprietà **ACID** (Atomicità, Consistenza, Isolamento, Durabilità).
- I meccanismi di **concorrenza** (più utenti accedono agli stessi dati senza conflitti).
- Il **controllo degli accessi** (autenticazione, autorizzazioni, ruoli).

4. Strumenti per amministrare la base di dati

Funzionalità dedicate ai **DBA (DataBase Administrator)**:

- Monitoraggio delle prestazioni.
- Backup e ripristino.
- Gestione della sicurezza.
- Ottimizzazione delle query.

5. Strumenti per sviluppare applicazioni su basi di dati

Il DBMS fornisce API, driver e interfacce che permettono a sviluppatori di integrare il database con applicazioni software:

- **ODBC/JDBC**, librerie in linguaggi come Python, Java, C#.
- Stored procedure e trigger.
- Funzionalità per generare report o connettersi a sistemi di Business Intelligence.

Tipi di linguaggi principali

1. **DDL (Data Definition Language)**
   - Serve a definire la struttura della base di dati.
   - Esempi: `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`.
   - Non manipola dati, ma crea/aggiorna lo **schema**.
2. **DML (Data Manipulation Language)**
   - Permette di leggere e modificare i dati.

- Esempi: `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
- È volutamente semplice, senza costrutti di controllo complessi: la logica del programma rimane fuori, nel linguaggio ospite.

Modalità d'uso in base al tipo di utente

- **Interfacce grafiche:** per chi non vuole scrivere codice (pensa a un gestionale aziendale o a strumenti come phpMyAdmin).
- **Linguaggi di interrogazione:** per utenti non programmatori ma che hanno dimestichezza con query SQL.
- **Linguaggi di programmazione integrati con DDL/DML** → per sviluppatori che devono costruire applicazioni. Qui si può:
  - Usare **precompilazione** o **estensioni del compilatore** (es. embedded SQL in C, chiamato Pro*C).
  - Usare API standard (ODBC/JDBC).
- **Linguaggi procedurali ad hoc** → tipo **PL/SQL** (Oracle) o **T-SQL** (SQL Server), che estendono SQL con costrutti di programmazione (cicli, condizioni, procedure).
- **Interfacce amichevoli** → maschere, report, strumenti di BI, che nascondono completamente il linguaggio testuale.

1. SQL **interattivo** (standard)

È il linguaggio dichiarativo che usi per **interrogare** e ottenere dati, senza occuparsi di come il DBMS li recupera.

```
SELECT Corso, Aula, Piano
FROM Aule, Corsi
WHERE NomeAula = Aula
  AND Piano = 'Terra';
```

risultato

```
Corso     Aula   Piano
Reti      N3     Terra
Sistemi   N3     Terra
```

Qui stai chiedendo: “Dammi corsi, aule e piani per tutti i corsi che si tengono al piano Terra”.

Il DBMS risolve la join tra **Aule** e **Corsi** usando la condizione `NomeAula = Aula`.

Domanda che emerge: *DS3 dove si trova?*

```
SELECT Piano
FROM Aule
WHERE NomeAula = 'DS3';
```

Risposta: Terra
Così sai che il corso *Basi di dati* di Rossi si tiene in **DS3**, che si trova al piano **Terra**.

2. SQL **ad-hoc procedurale** (PL/SQL di Oracle)

SQL da solo non ha cicli, condizioni o variabili: è “dichiarativo”.

Per logica più complessa i DBMS offrono linguaggi procedurali integrati (PL/SQL, T-SQL, PL/pgSQL...), che uniscono SQL a costrutti di programmazione.

```
DECLARE
   Stip NUMBER;
BEGIN
   SELECT Stipendio
   INTO Stip
   FROM Impiegato
   WHERE Matricola = '575488'
   FOR UPDATE OF Stipendio;
```

```
    IF Stip > 30 THEN
       UPDATE Impiegato
       SET Stipendio = Stipendio * 1.1
       WHERE Matricola = '575488';
    ELSE
       UPDATE Impiegato
       SET Stipendio = Stipendio * 1.15
       WHERE Matricola = '575488';
    END IF;

    COMMIT;

 EXCEPTION
    WHEN no_data_found THEN
       INSERT INTO Errori VALUES('Non esiste la matricola specificata');
 END;
```

Qui succede:

1. Legge lo stipendio dell'impiegato con matricola `575488`.
2. Se è maggiore di 30 → aumenta del 10%.

   Se è minore/uguale a 30 → aumenta del 15%.
3. Salva la modifica (`COMMIT`).
4. Se la matricola non esiste → scrive un record di errore.

Differenza chiave

- **SQL standard** → ottimo per interrogare dati o fare modifiche semplici.
- **PL/SQL (o simili)** → indispensabile quando serve logica condizionale, loop, gestione di eccezioni e variabili locali.

Livelli di descrizione dei dati

Livello Logico (Schema logico)

- Descrive la **struttura concettuale** dei dati e le relazioni tra loro, indipendentemente dalla memorizzazione fisica.
- **Esempio:**

  Studenti(Matricola char(8), Nome char(20), login char(8), AnnoNascita int, Reddito real)
  Corsi(IdeC char(8), Titolo char(20), Credito int)
  Esami(Matricola char(8), IdeC char(8), Voto int)

Livello Fisico (Schema fisico o interno)

- Descrive **come i dati vengono organizzati e memorizzati fisicamente** e quali strutture dati ausiliarie usare.
- **Esempio:**
  - Relazioni *Studenti* ed *Esami* organizzate in modo seriale.
  - *Corsi* organizzata in sequenza con indice.
  - Indice su `Matricola`, `(Matricola, IdeC)`.

Livello Esterno (Schema esterno o vista)

- Descrive **come appare la base di dati ad una certa applicazione o utente**.
- Ogni vista mostra solo una parte dello schema logico.
- **Esempio:**

  InfCorsi(IdeC char(8), Titolo char(20), NumEsami int)

Indipendenza Logica e Fisica

L’approccio con i **tre livelli di descrizione dei dati** (esterno, logico, fisico) è stato proposto per garantire due proprietà fondamentali dei DBMS:

- **Indipendenza logica**
  - I programmi applicativi **non devono essere modificati** se cambia lo **schema logico** (struttura concettuale della base dati).
  - Esempio: aggiungere un nuovo attributo a una tabella non deve rompere i programmi che usano solo gli attributi già esistenti.
- **Indipendenza fisica**
  - I programmi applicativi **non devono essere modificati** se cambia l’**organizzazione fisica** dei dati.
  - Esempio: spostare i dati da una rappresentazione sequenziale a una indicizzata non deve richiedere modifiche alle query.

Meccanismi per il Controllo delle Basi di Dati

I DBMS offrono meccanismi per garantire tre proprietà fondamentali:

1. **Integrità**
   - Mantenimento delle proprietà specificate nello schema tramite **vincoli di integrità**.
   - Esempio: chiave primaria unica, vincolo di non null, vincoli referenziali tra tabelle.
2. **Sicurezza**
   - Protezione dei dati da accessi non autorizzati e controllo delle operazioni consentite agli utenti.
   - Esempi di restrizioni:
     1. Alcuni utenti possono accedere solo a certi dati.
     2. Alcuni utenti possono leggere ma non modificare dati.
     3. Accesso limitato solo ai dati personali degli utenti.

4. Accesso parziale: modifiche consentite solo a dati anagrafici.
5. Restrizioni su uffici, orari e tipi di dati.
6. Possibilità di eseguire solo operazioni statistiche, senza accedere ai singoli dati.

3. **Affidabilità**
    - Protezione dei dati da **malfunzionamenti hardware o software**, fallimenti di transazioni, di sistema o disastri, e da interferenze dovute all'accesso concorrente.

Affidabilità nelle BD: Le Transazioni

- **Transazione:** sequenza di azioni di lettura/scrittura in memoria permanente e di elaborazioni in memoria temporanea, con proprietà fondamentali:
    1. **Atomicità:** o la transazione termina completamente o i suoi effetti vengono annullati (no effetti parziali).
    2. **Persistenza (Durabilità):** le modifiche completate sono permanenti anche in caso di malfunzionamenti.
    3. **Serializzabilità:** esecuzioni concorrenti hanno effetto equivalente a una esecuzione seriale.

Malfunzionamento (Failure)

Evento che può lasciare la base di dati in uno stato scorretto.

- Procedure di **recovery** vengono attivate quando l'anomalia è rilevata.
- Tipi di malfunzionamenti:
    1. **Fallimenti di transazione (transaction failure):** nessuna perdita di dati.
    2. **Fallimento di sistema (system failure):** la memoria permanente sopravvive.
    3. **Disastri (media/disk failure):** memoria permanente danneggiata.

Vantaggi e Svantaggi del DBMS

Vantaggi

- **Riduzione della ridondanza dei dati:** dati centralizzati e condivisi.
- **Integrità dei dati:** vincoli e regole per mantenere dati coerenti.
- **Sicurezza:** controllo degli accessi e restrizioni personalizzate.
- **Affidabilità:** gestione delle transazioni, rollback, recovery in caso di guasti.
- **Accesso concorrente:** più utenti possono usare i dati simultaneamente senza conflitti.
- **Indipendenza dei dati:** modifiche fisiche o logiche non impattano i programmi applicativi.
- **Supporto a linguaggi e applicazioni:** integrazione con software esterno tramite API o linguaggi procedurali (PL/SQL, T-SQL).

Svantaggi

- **Costo elevato:** licenze commerciali, hardware, personale specializzato.
- **Complessità:** progettazione e manutenzione richiedono competenze avanzate.
- **Prestazioni:** in sistemi molto grandi, gestione di query complesse può diventare lenta se non ottimizzata.
- **Dipendenza da software:** cambiamenti o migrazione tra DBMS diversi possono essere complicati.
- **Overhead amministrativo:** backup, sicurezza, tuning richiedono risorse e tempo.